CS-104 Quiz 1 (ANSWER KEY)

Fall 2026

Name: _____

This is an in-class, closed-book quiz (30 points). If you have any questions, please feel free to ask them. When you are finished, you can turn it in and go.

1. (5 points)

Write a code segment that builds a location code. It inputs a city, a two-letter state code, and a ZIP code (keep the ZIP as a string), and then prints a code made of:

- the first letter of the city,
- followed by a dash (`-`) and the state code,
- followed by the **last two** digits of the ZIP code (use indexing),

all in uppercase. Example run:

```
City: Grand Rapids
State: mi
ZIP: 49546
Code: G-MI46
```

Just write a code segment; you don't need to include documentation, an algorithm, or test cases.

```
city = input("City: ")
state = input("State: ")
zip_code = input("ZIP: ")
code = (city[0] + "-" + state + zip_code[-2] + zip_code[-1]).upper()
print("Code:", code)
```

Grading: three inputs 1, `city[0]` 1, dash and state 1, `zip_code[-2] + zip_code[-1]` (or `zip_code[3] + zip_code[4]`) 1, uppercase 1.

2. (4 points)

Write one Python statement for each task below (1 point each).

Assume `stock = {'pens': 30, 'pads': 7}`.

a) Add a new entry mapping `'clips'` to `50`.

b) Decrease the number of pens by 4, using its current value (don't just write 26).

c) Remove the entry for `'pads'` completely.

d) Print how many entries `stock` has.

a) `stock['clips'] = 50` **b)** `stock['pens'] = stock['pens'] - 4` **c)** `del stock['pads']` **d)** `print(len(stock))`

3. (6 points)

Write a code segment that inputs a number of total minutes from the user (as an integer) and prints how many whole days, remaining hours, and remaining minutes that is (a day has 1440 minutes). Here is an example run:

```
Total minutes: 3005
2 days, 2 hours, 5 minutes
```

Note that you should:

- implement the prompt as shown.
- use `//` and `%` as appropriate.
- print the result in the format shown above.

Just write a code segment; you don't need to include documentation, an algorithm, or test cases.

```
total = int(input("Total minutes: "))
days = total // 1440
hours = (total % 1440) // 60
minutes = total % 60
print(days, "days,", hours, "hours,", minutes, "minutes")
```

Grading: input/int conversion 1, days 1, hours 2, minutes 1, output format 1.

4. (6 points)

For each code segment below, identify the error and briefly explain how to fix it (2 points each: 1 for the error, 1 for the fix).

a)

```
print('I can't stop coding!')
```

b)

```
a = 2
b = 3
c = -4
x = 1
y = ax**2 + bx + c
```

c)

```
age = input("Enter your age")
age = age + 1
print(age)
```

a) The `'` in "can't" ends the string early (SyntaxError): use double quotes outside, or `\'`. b) `ax` and `bx` are read as variable names (NameError): write `a*x` and `b*x`. c) `input()` returns a string, so `+ 1` fails (TypeError): use `int(input(...))`.

5. (5 points)

Give the value of each Python expression below (½ point each). Make the resulting type clear (e.g., `8` vs. `8.0` vs. `'8'`). If it raises an error instead, name the error (e.g., "IndexError").

```
>>> 'x' + 'y'
```
'xy' — example answer

```
>>> 23 // 4
```

```
>>> 23 % 4
```

```
>>> 2 ** 2 ** 3
```

```
>>> 3 * 2.0
```

```
>>> 'xy' * 2
```

```
>>> '3' + '4'
```

```
>>> int(5.99)
```

```
>>> 'coding'[-3]
```

```
>>> len('code')
```

```
>>> 'age' + 21
```

Answers: `5`, `3`, `256`, `6.0`, `'xyxy'`, `'34'`, `5`, `'i'`, `4`, `TypeError`.

6. (4 points)

Consider the code below. What do the three `print` statements output?

```
x = ['m', 'n', 'o']
x[1] = 'q'
x.append('p')
y = x[0] + x[1]
x[-1] = 'r'
print(x)
print(y)
print(len(x))
```

`['m', 'q', 'o', 'r']`**, then** `'mq'`**, then** `4`**. Grading: 1 point per line, plus 1 point for all three correct.**